

Exercise 1

Create a new Java project. Create a class named Book (in the same file as Main is fine — no package is required), containing fields isbn (String), title (String), author (String), yearPublished (int), and price (double).

Provide a no-argument constructor and a parameterized constructor that initializes all fields. Create a display() method that prints all book information to the console in a readable format.

In the Main class, create an array to store the book data, you can use array/array list

Display the following menu repeatedly until the user chooses to exit:

===== Book Management Menu =====

1. Add new Book
2. Display all books
3. Search book by ISBN
4. Search book by name
5. Exit

Choose an option:

Implement all the functions listed in the menu.

Exercise 2 — Static Arrays

Using a static `int [ ]` array in Java, implement the following operations:

- Create a static array of 10 elements and assign a value to each element.
- Print all elements in the array.
- Find the maximum/minimum value in the array.
- Calculate the sum and average of the elements.
- Add an element to the end of the array (if there is still room).
- Remove the element at any given position `i`.
- Insert an element `x` at position `i`.
- Perform a linear search for an element `x` in the array.
- Rotate the array left/right by `k` positions.
- Reverse the array.
- Check whether the array is a palindrome (symmetric).
- Sort the array in ascending order (bubble sort, selection sort).

Exercise 3 — Dynamic Arrays (ArrayList)

Using `ArrayList<Integer>` in Java, implement the following operations:

- Create an `ArrayList` and add 5 integers to it.
- Print the elements.
- Access the element at any given position.
- Add an element at the beginning/end/a specific position.

- Remove an element by value/position.
- Check whether an element exists in the list.
- Count how many times an element appears.
- Filter even/odd elements into a new ArrayList.
- Merge two ArrayLists into one.
- Find the most frequently occurring element.
- Remove duplicate elements.

Exercise 4 — Array Statistics

Write a program to perform the following tasks on an integer array:

1. Calculate the average value of all elements in the array.
2. Find the second largest element in the array.
3. Count how many elements in the array are greater than a given threshold.

Input: An integer array and a threshold value (integer).

Output: - The average value of all elements in the array. - The second largest element in the array. - The count of elements greater than the threshold.

Exercise 5 — Product & Sales Management

1. Create a Product class with the structure:

- id (Integer)
- name (String)
- price (Float)
- quantity (Integer)

2. Create an Order class with the structure:

- id (Integer)
- quantity (Integer)
- date (LocalDateTime)

Represents a product and its ordered quantity.

Create a Main class (all classes above may be placed together in a single .java file — no package is required) to run the program with the requirement that all fields and methods of the Main class must be **non-static (concrete)**.

Create a program header menu:

Product Management Program

1. Enter Data | 2. Low-Stock Report | 3. Sort by Price | 4. Sell Product | 5. Sales Report | 6. Exit

The menu is shown when the program starts.

Entering the product list

If the user selects option 1, the program does the following:

- Asks the user to enter the total number of products to input.
- Collects the information for each product.
- Validates the input against these requirements:

$0 \leq \text{price} \leq 10,000,000$

$0 \leq \text{quantity}$

When the user selects option 2, display a list, sorted from smallest to largest, of the 3 products with the lowest stock quantity.

When the user selects option 3, display the list sorted by price in descending order.

When the user selects option 4: Display the product list, ask the user to enter the ID of the product to sell and a quantity, with the requirements:

- The ID must match an existing product's ID.
- The quantity must be less than or equal to the product's current stock quantity.

After the sale is completed, subtract the sold quantity from that product's stock.

When the user selects option 5, display the sales history.

When the user selects option 6, exit the program.

Exercise 6 — Electric Bill Management

To manage electricity billing receipts, the following information is needed:

- For each receipt: the customer household, the previous meter reading, the new meter reading, and the amount due.
- Information specific to each household: the head of household's name, house number, and electric meter ID.

Requirement 1: Build a `Customer` class to store the information specific to each household.

Requirement 2: Build a `Bill` class to manage electricity usage and payment for the households.

Requirement 3: Build methods to add, remove, and edit the information specific to each household.

Requirement 4: Write a method to calculate the electricity bill for each household using the formula: $(\text{new reading} - \text{old reading}) \times 5$.

Exercise 7 — Library Borrowing Card Management

A university's central library needs to manage book borrowing and returns. Students register to borrow books using a library card.

Each card stores the following information: borrowing slip ID, borrow date, due date, book ID, and the personal information of the borrowing student. The student's personal information includes: full name, age, and class.

To keep the console application simple, we treat the borrow date and due date as positive integers.

Requirement 1: Build a `Student` class to manage each student's information.

Requirement 2: Build a `BorrowCard` class to manage students' book borrowing and returns.

Requirement 3: Build methods to add, remove (by borrowing slip ID), and display the information of the borrowing cards.

Exercise 8 — Cafe Management System

1. Create the product data structure:

Create a class `Product` with the following attributes: - `id` (int) - `name` (String, maximum 20 characters) - `price` (float)

2. Create the booking data structure:

Create a class `Booking` with the following attributes: - `id` (int) - `productId` (int) - `productName` (String, maximum 20 characters) - `price` (float) - `date` (String, format 'dd-mm-yy') - `quantity` (int)

This class models a booking/order placed at the cafe.

3. Create the program's header menu:

Cafe Management Program

1. Enter Product | 2. Place Booking | 3. Find Booking by Date (dd-mm-yy) | 4. Product List | 5. Booking History | 6. Exit

Enter product feature: - If the user selects option 1, the program should: - Ask the user for the total number of products to enter. - Enter the detailed information for each product. - Validate the entered value: price must be between 0 and 100,000. - After entry, automatically save the products to the in-memory data list.

Place booking feature: - If the user selects option 2, start the booking feature. It should: - Ask for the product ID for the order. - Ask for the quantity of the product. - Store the order in the corresponding data list.

Find booking by date feature: - If the user selects option 3, the program allows searching for bookings by date (the user enters a date in the correct format to search).

Display product list feature: - If the user selects option 4, display the list of products, sorted by name and price columns.

Display booking history feature: - If the user selects option 5, display the booking history. - If the user selects option 6, exit the program.